COMPUTER LABORATORY – I

K-Means Clustering on the Iris Dataset

Beginner-friendly practical, theory and viva preparation guide

EXPERIMENT 7 – CLUSTERING ANALYSIS
Implement K-Means on Iris.csv and estimate the number of clusters using the Elbow Method.

What you will learn

- Explain every cell and important line in the uploaded notebook.
- K-Means, centroids, distance, inertia and the Elbow Method.
- Why standardization is needed and why Species is excluded.
- Pandas, scikit-learn and Matplotlib functions and parameters.
- Corrected code, graph interpretation, theory viva and code cross-questions.

Prepared from the uploaded Assignment_7(A).ipynb and Computer Laboratory-I assignment list. The assignment sheet identifies this as Clustering Analysis (page 3, item 7).

1. Problem statement and Iris dataset

The task is to group similar Iris flowers using their measurements. We do not provide the actual species to the clustering algorithm. Instead, the algorithm tries to discover groups from the measurements. This is unsupervised learning.

What is the Iris dataset?

Iris is a commonly used introductory machine-learning dataset. A standard version contains 150 flower observations, four numeric measurements and a species category. Your notebook reads a local file named iris.csv. The code expects the column names shown below; check iris.columns if your CSV differs.

Column	Meaning	Role
SepalLengthCm	Sepal length in cm	Input feature
SepalWidthCm	Sepal width in cm	Input feature
PetalLengthCm	Petal length in cm	Input feature
PetalWidthCm	Petal width in cm	Input feature
Species	Actual species category	Excluded label

Why remove Species?

Species is the answer we might want to compare against after clustering. K-Means should not be given this answer during training. Including it would cause label leakage and it is categorical rather than a normal numeric measurement.

Essential vocabulary

Observation: one flower/row. Feature: a measurement column. Feature matrix X: rows are flowers and columns are measurements. Cluster: a group of similar points. Centroid: the mean centre of a cluster. Cluster label: an integer such as 0, 1 or 2 assigned to a row.

Important: Cluster 0 does not automatically mean Setosa. Cluster IDs are arbitrary and are not species names.

2. K-Means: concept and algorithm

K-Means is an unsupervised clustering algorithm. It divides observations into K groups so that points are close to the centre of their assigned group. K is chosen before the final model is trained.

Working steps

- Choose K, the desired number of clusters.
- Initialize K centroids. scikit-learn uses k-means++ by default to choose sensible starting centres.
- Measure each point's distance from the centroids.
- Assign each point to its nearest centroid.
- Recalculate each centroid as the mean of the points assigned to it.
- Repeat assignment and centroid update until the centres/assignments stop changing significantly or an iteration limit is reached.

Euclidean distance

The usual straight-line distance between two feature vectors is:

$d(x,c) = \sqrt{[(x_1-c_1)^2 + (x_2-c_2)^2 + \ldots + (x_n-c_n)^2]}$

Here x is one flower and c is a centroid. Iris has four input features, so the distance is calculated across four standardized measurements. The nearest centroid wins.

Centroid

A centroid is the average location of all points currently assigned to a cluster. For each feature separately, add the values in that cluster and divide by the number of points.

Advantages

- Simple and relatively fast; easy to explain.
- Works well when groups are compact and reasonably separated.
- Can be applied to many numeric observations.

Limitations

- K must be chosen in advance.
- Sensitive to feature scales, outliers and initial centroids.
- May not work well for curved/irregular groups or clusters with very different densities.

3. StandardScaler, inertia and Elbow Method

Why standardize?

K-Means relies on distance. If one feature has a much larger numeric range, it can dominate the distance even when it is not more important. StandardScaler puts measurements on a comparable scale.

$z = (x - \mu) / \sigma$
x = original value; μ = feature mean; σ = feature standard deviation. After standardization, each feature is approximately centred at 0 with standard deviation 1.

What is inertia?

Inertia is the sum of squared distances between every point and the centroid of the cluster it belongs to. It measures how tightly the points fit their own centres. Smaller inertia means points are closer to their centroids. Inertia is non-negative.

Why does inertia fall when K increases?

More clusters mean more centroids, so each point can usually be assigned to a closer centre. Therefore, the best possible inertia cannot increase when K is increased. But using a very large K can simply create many tiny groups, so the lowest inertia is not automatically the best model.

Elbow Method

Fit K-Means several times for K = 1, 2, ..., 10. Record each model's inertia. Plot K on the x-axis and inertia on the y-axis. The curve usually drops quickly at first and then flattens. Choose a K near the bend where adding another cluster gives a much smaller improvement. This bend is called the elbow.

For your notebook: K values 1 to 10 are tested. The final code then uses K = 3. You should visually inspect your generated elbow graph; the code by itself does not prove that 3 is the best K.

Silhouette score – extra theory

Silhouette score compares how close a point is to its own cluster against other clusters. It ranges from −1 to +1. Near +1 is a good assignment; near 0 indicates overlap; a negative score may indicate a point is closer to another cluster. Your notebook does not calculate this score.

4. Code explanation: imports and data preparation

Cell 1 – imports

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

pandas as pd: tabular data handling; used for read_csv(), head() and drop(). NumPy as np: numerical arrays and maths; imported but not used directly. train_test_split: divides data into train/test subsets; imported but unused in this clustering notebook. KMeans: the clustering estimator. StandardScaler: standardizes numeric features. matplotlib.pyplot as plt: plotting functions.

Cell 2 – read the CSV

```
iris = pd.read_csv('iris.csv')
iris.head()
```

pd.read_csv() reads a comma-separated file and returns a pandas DataFrame. The path is relative to the notebook's current working directory. iris.head() displays the first five rows by default. It is a quick check of whether the data loaded and the columns look correct. It does not modify the data.

Cell 3 – choose features and scale them

```
X = iris.drop(columns='Species')
X_scaled = StandardScaler().fit_transform(X)
```

drop(columns='Species') returns a copy without the Species column. X now contains the four measurement columns. StandardScaler() creates the scaler. fit_transform(X) learns each column's mean and standard deviation, then applies the z-score formula. X_scaled is a numeric array that is passed to K-Means.

Why no scaling of Species? It is a category/answer, not a numeric measurement used to calculate flower similarity.

5. Code explanation: Elbow loop and plot

Cell 4 – calculate inertia

```
inertias = []
for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertias.append(model.inertia_)
```

inertias = [] creates an empty list. range(1, 11) produces 1 through 10; 11 is excluded. The loop trains a separate model for every K.

n_clusters=k sets the number of clusters for this iteration. random_state=42 fixes the random seed so results are reproducible in the same setup. 42 is not special. n_init=10 runs the algorithm 10 times with different initial centroid choices and keeps the best result by inertia. fit(X_scaled) trains the model. model.inertia_ is the fitted model's inertia value. append() adds that number to the list.

Draw the Elbow curve

```
plt.plot(range(1, 11), inertias, marker='o')
plt.xlabel('Number of clusters')
plt.xticks(range(1, 11))
plt.show()
```

plot(x,y) draws the line graph. marker='o' puts a circle at each data point. xlabel() names the horizontal axis. xticks() sets the x-axis tick positions. show() displays the graph.

Bug in uploaded notebook: it says `plt.xsticks(range(1,11))`. The correct Matplotlib function is `plt.xticks(range(1,11))`. xsticks is not a pyplot function and will raise an AttributeError.

Graph explanation: x-axis = K (number of clusters); y-axis = inertia. Look for the bend after which inertia decreases more slowly. Do not invent exact inertia values; run the notebook to obtain them.

6. Code explanation: final clustering and visualization

```
k = 3
model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(X_scaled)

plt.scatter(iris['PetalLengthCm'],
            iris['PetalWidthCm'],
            c=labels, cmap='viridis')
plt.xlabel('Petal length (cm)')
plt.ylabel('Petal width (cm)')
plt.title('Iris K-Means clusters')
plt.show()
```

k = 3 stores the selected cluster count. The next line creates a KMeans model with three clusters. In cleaner code, use `n_clusters=k` rather than repeating the number 3.

fit_predict(X_scaled) both trains the model and returns one integer cluster label for each flower. Labels are generally 0, 1 and 2 when K = 3. They are not actual species names.

plt.scatter(x,y) draws one point per flower. PetalLengthCm is the x-coordinate and PetalWidthCm is the y-coordinate. c=labels colours points according to their assigned cluster. cmap='viridis' chooses the colour map. xlabel(), ylabel() and title() describe the axes and chart.

Important cross-question: Why are only two columns plotted if four columns were used for clustering? Because a normal scatter plot is two-dimensional. K-Means was trained using all four standardized features; the chart displays only two of them so people can inspect the groups visually.

Improved final code

```
# Use the selected k variable consistently
k = 3
final_model = KMeans(
    n_clusters=k, random_state=42, n_init=10
)
labels = final_model.fit_predict(X_scaled)
```

The notebook's last cell is empty. It has no effect on the result.

7. Complete corrected notebook code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Load data
iris = pd.read_csv("iris.csv")
print(iris.head())

# Remove the known species label
X = iris.drop(columns=["Species"])

# Standardize the four numeric features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow Method: test K from 1 to 10
inertias = []
k_values = range(1, 11)

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(list(k_values), inertias, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.xticks(list(k_values))
plt.grid(True, alpha=0.3)
plt.show()

# Choose K after inspecting the elbow
k = 3
final_model = KMeans(
    n_clusters=k, random_state=42, n_init=10
)
labels = final_model.fit_predict(X_scaled)

# Visualize using two of the four features
plt.scatter(iris["PetalLengthCm"],
            iris["PetalWidthCm"],
            c=labels, cmap="viridis")
plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")
plt.title("Iris K-Means clusters")
plt.show()
```

What was improved?

- Removed unused NumPy and train_test_split imports.
- Corrected xsticks() to xticks().
- Used the variable k in the final KMeans constructor.
- Added inertia y-axis label, chart title and grid to the Elbow graph.
- Stored StandardScaler in a variable so fit and transform are visible as separate concepts.

How to interpret the scatter plot

Each dot is one flower. Its horizontal position is petal length and vertical position is petal width. The colour represents the cluster assigned by the model. Similar-looking colour groups suggest the model found structure in the measurements. Some clusters may overlap; clustering does not guarantee perfect separation.

The actual output depends on the iris.csv file and installed library version. Your uploaded notebook contains code but no saved graph/output values, so run it to confirm the exact elbow and plot.

8. Theory viva questions and answers

Q. What is machine learning?

Answer: A way for computers to learn patterns from data to make predictions or decisions.

Q. Name the main types of machine learning.

Answer: Supervised learning, unsupervised learning and reinforcement learning.

Q. What is supervised learning?

Answer: Learning from labelled examples where the correct output is provided.

Q. What is unsupervised learning?

Answer: Finding structure in data without target labels. K-Means is an example.

Q. Why is K-Means unsupervised?

Answer: It discovers groups from measurements without using the Species column as a target.

Q. Why is it called K-Means?

Answer: K is the number of groups; each group is represented by the mean location of its points.

Q. What is a centroid?

Answer: The average position of all points assigned to a cluster.

Q. How is a centroid updated?

Answer: Calculate the mean of each feature among points in that cluster.

Q. What distance does K-Means commonly use?

Answer: Euclidean distance in standard scikit-learn K-Means.

Q. What is inertia?

Answer: The sum of squared distances from each point to its assigned cluster centre.

Q. Why does inertia decrease as K grows?

Answer: More centres give points more nearby choices, so the best fit cannot get worse.

Q. What is the Elbow Method?

Answer: A heuristic that chooses K near the bend where adding clusters gives diminishing improvement.

Q. Is the elbow always clear?

Answer: No. Some datasets have no obvious bend; use other metrics and domain understanding too.

Q. Why standardize before K-Means?

Answer: Distance-based methods are affected by feature scale; scaling avoids one large-range feature dominating.

Q. Standardization vs normalization?

Answer: Standardization usually gives mean 0 and standard deviation 1. Min-max normalization often rescales values to 0–1.

Q. What is an outlier?

Answer: A value far from most observations; it can pull the mean centroid and affect K-Means.

Q. What is a cluster label?

Answer: An integer ID assigned to a point, such as 0, 1 or 2.

Q. Does cluster 0 mean Setosa?

Answer: No. Cluster IDs are arbitrary and may be permuted.

Q. What are K-Means limitations?

Answer: It requires K, is sensitive to scale/outliers/initialization, and favours compact roughly spherical groups.

Q. What is silhouette score?

Answer: A measure of how well points fit their own cluster compared with other clusters; it ranges from −1 to +1.

9. Code cross-questions and last-minute revision

Q. What does import pandas as pd mean? Import pandas and refer to it using pd.
Q. What is a DataFrame? A two-dimensional table with labelled rows and columns.
Q. What does read_csv() return? A pandas DataFrame.
Q. What does head() do? Shows the first five rows by default; head(10) shows ten.
Q. What does drop(columns='Species') do? Returns a copy without that column; the original is unchanged by default.
Q. fit() vs fit_transform()? fit() learns parameters; fit_transform() learns and applies the transformation.
Q. What does StandardScaler learn? The mean and standard deviation of each feature.
Q. Why random_state=42? To make randomized initialization repeatable; 42 is an arbitrary seed.
Q. What is n_init=10? Run 10 initializations and keep the best according to inertia.
Q. What does range(1,11) produce? Integers 1 through 10; the stop value is excluded.
Q. What does append() do? Adds an item to the end of a list.
Q. What is model.inertia_? The fitted model's within-cluster sum of squared distances.
Q. fit_predict() vs fit()? fit() trains; fit_predict() trains and returns labels.
Q. What does c=labels do in scatter()? Uses cluster IDs to colour the points.
Q. What does cmap='viridis' do? Selects the colour map for numeric labels.
Q. Why is train_test_split unused? This workflow does not split data for supervised prediction; remove the unused import.
Q. Why is NumPy unused? No np function is called directly in the notebook.
Q. What if iris.csv is missing? A FileNotFoundError; check the working directory or file path.
Q. What if Species has different capitalization? Column names must match exactly; inspect iris.columns.
Q. What does xsticks() do? It is a typo; use plt.xticks() instead.
Q. Why only two features in the scatter? The chart is 2D; clustering still used all four standardized features.
Q. How compare clusters with species? Compare labels with Species after fitting, not as an input to K-Means.

30-second experiment explanation

“I used K-Means, an unsupervised clustering algorithm, on the four numerical Iris measurements. I removed Species so the model could discover groups without being given the answer. Because K-Means uses distances, I standardized the features. I tested K values from 1 to 10 and recorded inertia for the Elbow plot. I then used K = 3, obtained cluster IDs using fit_predict(), and visualized them using petal length and width.”

Before the viva – remember

• Exclude Species from training; use it only for later comparison.
• Scaling matters because K-Means is distance-based.
• K = number of clusters; centroid = cluster mean; inertia = within-cluster squared distance.
• Elbow = bend in K versus inertia curve.
• fit() trains; fit_predict() trains and returns labels; fit_transform() learns and transforms.
• Correct typo: plt.xsticks() → plt.xticks().
• Cluster numbers are not species names.

Source note: Assignment sheet page 3, item 7 specifies K-Means on Iris and the Elbow Method. Code explanations follow the six cells in Assignment_7(A).ipynb.